

Sql Questions

Generated on February 05, 2026 at 01:29 PM | 160 entries

Q1. What is SQL and why is it important?

[EASY] | *Category: Basics* | *Type: theory*

Answer: SQL (Structured Query Language) is used to manage and manipulate relational databases. It allows developers to create, read, update, and delete data efficiently. It's essential because almost all modern applications rely on structured data storage and retrieval.

Q2. What is the difference between SQL and MySQL?

[EASY] | *Category: Basics* | *Type: theory*

Answer: SQL is a language for querying databases, whereas MySQL is a database management system that implements SQL. In short, SQL is the language; MySQL is a tool that uses it to manage databases.

Q3. What are the different types of SQL commands?

[EASY] | *Category: Basics* | *Type: theory*

Answer: SQL commands are grouped into categories: DDL (Data Definition Language), DML (Data Manipulation Language), DCL (Data Control Language), TCL (Transaction Control Language), and DQL (Data Query Language). Each type performs a unique database operation.

Q4. What is the difference between WHERE and HAVING clauses?

[MEDIUM] | *Category: Filtering* | *Type: theory*

Answer: WHERE filters rows before aggregation, while HAVING filters groups after aggregation. You use WHERE with raw data and HAVING with aggregated results like SUM or COUNT, often combined with GROUP BY.

Q5. Explain primary key and foreign key.

[EASY] | *Category: Constraints* | *Type: theory*

Answer: A primary key uniquely identifies each record in a table. A foreign key establishes a link between two tables by referencing the primary key of another table, enforcing relational integrity between data sets.

Q6. What is normalization in SQL?

[MEDIUM] | *Category: Database Design* | *Type: theory*

Answer: Normalization organizes data to reduce redundancy and improve data integrity. It divides tables into smaller, related tables and uses relationships to maintain data consistency. The main forms include 1NF, 2NF, and 3NF.

Q7. What is denormalization?

[MEDIUM] | *Category: Database Design* | *Type: theory*

Answer: Denormalization combines tables to improve read performance. It introduces controlled redundancy to reduce complex joins during queries, often used in reporting or analytics databases where read speed is prioritized over updates.

Q8. What are joins in SQL?

[EASY] | *Category: Joins* | *Type: theory*

Answer: Joins combine rows from multiple tables based on related columns. Common joins include INNER, LEFT, RIGHT, and FULL JOIN. They allow complex queries across tables, enabling data relationships to be queried efficiently.

Q9. Explain INNER JOIN.

[EASY] | *Category: Joins* | *Type: theory*

Answer: INNER JOIN returns only the rows that have matching values in both tables. It's used when you want data that exists in both tables based on a common column.

Q10. Explain LEFT JOIN.

[EASY] | *Category: Joins* | *Type: theory*

Answer: LEFT JOIN returns all records from the left table and matched records from the right table. If no match exists, NULL values are shown for columns from the right table.

Q11. What is a subquery?

[MEDIUM] | *Category: Subqueries* | *Type: theory*

Answer: A subquery is a query nested inside another SQL query. It can return single or multiple values and is often used to filter, calculate, or compare data dynamically based on another query.

Q12. What are indexes in SQL?

[MEDIUM] | *Category: Optimization* | *Type: theory*

Answer: Indexes improve the speed of data retrieval by creating a data structure for quick lookups. They act like a book index, allowing faster access to rows based on column values but increase storage and write overhead.

Q13. What is the difference between clustered and non-clustered indexes?

[MEDIUM] | *Category: Optimization* | *Type: theory*

Answer: A clustered index sorts and stores the actual data rows in order, while a non-clustered index creates a separate structure pointing to data locations. A table can have only one clustered index.

Q14. What is a view in SQL?

[EASY] | *Category: Views* | *Type: theory*

Answer: A view is a virtual table created from a query. It stores no physical data but allows users to simplify complex queries, enhance security, and present specific data subsets.

Q15. Explain ACID properties.

[HARD] | *Category: Transactions* | *Type: theory*

Answer: ACID stands for Atomicity, Consistency, Isolation, and Durability — the four principles ensuring reliable transaction processing. They guarantee that transactions are processed completely or not at all, maintaining data integrity.

Q16. What is a transaction?

[MEDIUM] | *Category: Transactions* | *Type: theory*

Answer: A transaction is a set of operations that execute as a single unit. If any part fails, the entire transaction is rolled back to maintain data integrity. It's crucial in banking or financial systems.

Q17. What is the difference between DELETE, TRUNCATE, and DROP?

[MEDIUM] | *Category: Data Manipulation* | *Type: theory*

Answer: DELETE removes specific rows, TRUNCATE removes all rows but keeps the table structure, and DROP removes the entire table from the database. DELETE can use conditions; the others cannot.

Q18. What are constraints in SQL?

[EASY] | *Category: Constraints* | *Type: theory*

Answer: Constraints enforce rules on data in tables. Common ones are PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT. They maintain data accuracy and prevent invalid entries.

Q19. What is the difference between UNION and UNION ALL?

[MEDIUM] | *Category: Set Operations* | *Type: theory*

Answer: UNION combines results and removes duplicates, while UNION ALL includes duplicates. UNION requires extra processing for distinct filtering, whereas UNION ALL is faster since it skips duplicate checks.

Q20. What is a stored procedure?

[MEDIUM] | *Category: Procedures* | *Type: theory*

Answer: A stored procedure is a group of SQL statements stored in the database. It's reusable, improves performance, and adds security by reducing direct data manipulation through user queries.

Q21. What is a trigger in SQL?

[MEDIUM] | *Category: Triggers* | *Type: theory*

Answer: A trigger automatically executes predefined actions in response to specific table events like INSERT, UPDATE, or DELETE. It's useful for maintaining audit logs or enforcing data integrity.

Q22. What is a cursor?

[HARD] | *Category: Cursors* | *Type: theory*

Answer: A cursor allows row-by-row processing of result sets. It's helpful in cases where set-based operations aren't sufficient, though it's slower and less efficient compared to bulk SQL operations.

Q23. What are aggregate functions?

[EASY] | *Category: Functions* | *Type: theory*

Answer: Aggregate functions perform calculations on multiple rows and return a single value. Common examples include COUNT, SUM, AVG, MIN, and MAX. They're often used with GROUP BY clauses.

Q24. What are scalar functions?

[EASY] | *Category: Functions* | *Type: theory*

Answer: Scalar functions return a single value for each input value. Examples include UPPER(), LOWER(), LEN(), and ROUND(). They operate on individual data elements rather than groups.

Q25. What is the GROUP BY clause used for?

[EASY] | *Category: Grouping* | *Type: theory*

Answer: GROUP BY organizes rows into groups based on one or more columns. It's often used with aggregate functions to summarize data, such as calculating total sales per region.

Q26. What is the difference between IN and EXISTS?

[MEDIUM] | *Category: Subqueries* | *Type: theory*

Answer: IN compares a value to a list or subquery result, while EXISTS checks for the existence of rows in a subquery. EXISTS is generally faster for correlated subqueries.

Q27. What is a self-join?

[MEDIUM] | *Category: Joins* | *Type: theory*

Answer: A self-join joins a table with itself to compare rows within the same table. It's often used for hierarchical or recursive relationships, like employees reporting to managers.

Q28. What is the purpose of the DISTINCT keyword?

[EASY] | *Category: Filtering* | *Type: theory*

Answer: DISTINCT removes duplicate records from query results. It ensures that only unique combinations of selected columns appear in the output.

Q29. What are wildcards in SQL?

[EASY] | *Category: Filtering* | *Type: theory*

Answer: Wildcards are used with the LIKE operator for pattern matching. For example, % matches any sequence of characters, and _ matches a single character. Useful for partial searches.

Q30. What is a surrogate key?

[MEDIUM] | *Category: Keys* | *Type: theory*

Answer: A surrogate key is an artificial unique identifier for a record, often implemented as an auto-increment column. It has no business meaning but ensures uniqueness in relational models.

Q31. What is referential integrity?

[MEDIUM] | *Category: Constraints* | *Type: theory*

Answer: Referential integrity ensures consistency between related tables using foreign keys. It prevents actions that would leave orphan records or break relationships between tables.

Q32. What is the difference between CHAR and VARCHAR?

[EASY] | *Category: Data Types* | *Type: theory*

Answer: CHAR has a fixed length, while VARCHAR has a variable length. CHAR is faster for fixed-size data, while VARCHAR is more space-efficient for varying text lengths.

Q33. What is the COALESCE function?

[EASY] | *Category: Functions* | *Type: theory*

Answer: COALESCE returns the first non-null value among its arguments. It's used for handling NULL values and providing default outputs in queries.

Q34. What is a schema in SQL?

[MEDIUM] | *Category: Database Design* | *Type: theory*

Answer: A schema is a logical container grouping related database objects like tables, views, and procedures. It helps organize data and manage permissions efficiently.

Q35. What are transactions isolation levels?

[HARD] | *Category: Transactions* | *Type: theory*

Answer: Isolation levels define how transaction integrity is maintained when multiple users access data concurrently. Common levels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, and SERIALIZABLE.

Q36. How can you optimize SQL queries?

[HARD] | *Category: Optimization* | *Type: theory*

Answer: Query optimization involves indexing, avoiding SELECT *, minimizing joins, using WHERE clauses, and analyzing query plans. Proper normalization, caching, and partitioning also enhance performance.

Q37. What is the difference between RANK() and DENSE_RANK()?

[MEDIUM] | *Category: Window Functions* | *Type: theory*

Answer: Both assign rankings based on order, but RANK() skips numbers after ties, whereas DENSE_RANK() doesn't. If two rows tie for rank 1, the next RANK is 3, but DENSE_RANK is 2.

Q38. What is a window function in SQL?

[HARD] | *Category: Window Functions* | *Type: theory*

Answer: A window function performs calculations across a set of rows related to the current row. Examples: ROW_NUMBER(), RANK(), and SUM() OVER(). They're used for analytics without grouping data.

Q39. What is the difference between BETWEEN and IN?

[EASY] | *Category: Filtering* | *Type: theory*

Answer: BETWEEN filters a range of values (e.g., salary BETWEEN 2000 AND 5000), while IN matches specific values. BETWEEN works for continuous ranges; IN is for discrete value lists.

Q40. What is the CASE statement used for?

[MEDIUM] | *Category: Functions* | *Type: theory*

Answer: CASE adds conditional logic to SQL queries, similar to IF-ELSE. It allows returning specific values based on given conditions within SELECT or WHERE clauses.

Q41. What are temporary tables?

[MEDIUM] | *Category: Tables* | *Type: theory*

Answer: Temporary tables store intermediate data during query execution. They exist only during a session and help break complex operations into simpler steps.

Q42. What is a composite key?

[MEDIUM] | *Category: Keys* | *Type: theory*

Answer: A composite key combines two or more columns to uniquely identify each record. It's used when no single column can serve as a unique identifier.

Q43. What is a recursive query?

[HARD] | *Category: Subqueries* | *Type: theory*

Answer: A recursive query repeatedly executes until a condition is met. Commonly used for hierarchical data like employee-manager structures using WITH RECURSIVE or CTEs.

Q44. What are common table expressions (CTEs)?

[MEDIUM] | *Category: CTEs* | *Type: theory*

Answer: CTEs are temporary result sets defined using WITH. They simplify complex queries, improve readability, and can be recursive for hierarchical data traversal.

Q45. What are SQL injections?

[HARD] | *Category: Security* | *Type: theory*

Answer: SQL injection is a security vulnerability where attackers manipulate queries by injecting malicious input. Using parameterized queries and ORM frameworks helps prevent it.

Q46. What is a pivot table in SQL?

[HARD] | *Category: Aggregation* | *Type: theory*

Answer: A pivot table summarizes and transforms rows into columns. It's useful for aggregating and reporting data, often achieved using the PIVOT operator or conditional aggregation.

Q47. What is the difference between logical and physical data independence?

[HARD] | *Category: Database Design* | *Type: theory*

Answer: Logical independence allows changes in logical schema without affecting applications. Physical independence allows changes in storage without affecting the logical schema.

Q48. What is the difference between DELETE and UPDATE triggers?

[MEDIUM] | *Category: Triggers* | *Type: theory*

Answer: DELETE triggers activate when rows are removed, while UPDATE triggers execute when data changes. Both are used to maintain audit logs or synchronize tables.

Q49. What are materialized views?

[HARD] | *Category: Views* | *Type: theory*

Answer: A materialized view stores the result of a query physically. Unlike normal views, it requires refreshing to update data, improving query performance in analytics.

Q50. What are window partitions?

[HARD] | *Category: Window Functions* | *Type: theory*

Answer: Window partitions divide query results into groups before applying window functions. It allows calculations like ranking or running totals within specific data subsets.

Q51. How do you handle recursive queries without CTEs?

[HARD] | *Category: Hierarchical Queries* | *Type: query*

Answer: Pre-CTE recursion uses stored procedures with loops/temp tables: `DECLARE @level INT=0; CREATE TEMP \#hierarchy; INSERT \#hierarchy SELECT \* FROM nodes WHERE parent IS NULL; WHILE @@ROWCOUNT>0 BEGIN SET @level+=1; INSERT \#hierarchy SELECT n.\* FROM nodes n JOIN \#hierarchy h ON n.parent=h.id WHERE h.level=@level-1; END.` Iterative, avoids stack overflow but verbose. Suitable for Oracle pre-11g or limited depth. Perf: index parent; limit iterations. Modern preference: recursive CTEs for readability.

Q52. Explain the use of LATERAL joins.

[HARD] | *Category: Joins* | *Type: query*

Answer: LATERAL enables outer-dependent subqueries in FROM: `SELECT e.name, p.project FROM employees e CROSS JOIN LATERAL (SELECT \* FROM projects WHERE emp\_id=e.id ORDER BY start DESC LIMIT 3) p.` Like correlated but returns sets; unnest arrays/JSON. Postgres/Oracle support; for top-N per row. Pros: cleaner than scalar subqueries; cons: serial execution unless parallel. Index join cols for speed. In 2025, vital for app dev with variable results.

Q53. What are covering indexes and when to use them?

[HARD] | *Category: Performance* | *Type: query*

Answer: Covering indexes include all SELECT columns (INCLUDE clause): `CREATE INDEX idx\_cover ON sales(customer\_id, date) INCLUDE (amount, tax).` Query uses index only, no table access—faster I/O. Detect via EXPLAIN 'Using index'. Use for read-heavy reports; avoid on write-frequent tables (rebuild overhead). Composite keys + INCLUDE for non-keys. In tuning, prioritize over scans. SQL Server/Postgres; size trade-off.

Q54. How do you implement row-level security in SQL?

[HARD] | *Category: Security | Type: query*

Answer: RLS policies filter rows per user: Postgres CREATE POLICY emp_policy ON employees USING (dept=current_setting('app.current_dept')); Enforced transparently. SQL Server: CREATE SECURITY POLICY p WITH (STATE=ON) TO role; predicate function filters. Pros: DB-enforced, no app code; cons: perf hit (scan more). Test with SET ROLE; audit violations. For multi-tenant SaaS, combine VPD. 2025 compliance essential (GDPR).

Q55. What is the difference between correlated subquery and lateral join?

[HARD] | *Category: Subqueries* | *Type: query*

Answer: Correlated subquery in WHERE/SELECT executes per row: `SELECT \* FROM emp e WHERE sal > (SELECT AVG(sal) FROM peers p WHERE p.dept=e.dept)`. Lateral in FROM: `SELECT e.\*, sub.avg FROM emp e LEFT JOIN LATERAL (SELECT AVG(sal) FROM peers p WHERE p.dept=e.dept) sub ON true`. Lateral returns multi-cols/sets; often faster, parallelizable. Use lateral for complex per-row (e.g., arrays). Both correlated; index outer.

Q56. Explain dynamic pivoting in SQL.

[HARD] | *Category: Data Transformation* | *Type: query*

Answer: Dynamic pivot builds columns runtime: `DECLARE @cols NVARCHAR(MAX)=STUFF((SELECT ','+QUOTENAME(month) FROM months FOR XML PATH(''),1,1,''); EXEC('SELECT \* FROM sales PIVOT(SUM(amount) FOR month IN('+@cols+'))')`. For unknown pivots (e.g., dynamic categories). Sanitize to prevent injection; use `sp\_executesql` params. Alternatives: `STRING\_AGG` in Postgres. Common in dashboards; perf: pre-aggregate.

Q57. How do you detect and resolve deadlocks?

[HARD] | *Category: Transactions* | *Type: query*

Answer: Deadlocks: circular waits; detect via Extended Events (`deadlock\_graph`) or `sys.dm\_os\_waiting\_tasks`. Graph XML shows victims/processes. Resolve: short trans, lock hints (`UPDLOCK`), consistent order, isolation (`SNAPSHOT`). Prevention: app retries with exponential backoff. Analyze traces for patterns; Query Store historical. High-concurrency key; monitor `@@DEADLOCK\_INT`.

Q58. What is query hinting and when to use it?

[MEDIUM] | *Category: Performance* | *Type: query*

Answer: Hints force plans: MySQL `SELECT \* FROM t FORCE INDEX(idx)`; SQL Server `OPTION(LOOP JOIN)`. For bad cardinality estimates or missing stats. Use rarely—aspirin, not vitamin; fix root (stats/index). Non-portable; adaptive queries (SQL 2017+) reduce need. Analyze `EXPLAIN` first. In emergencies like prod slowness.

Q59. Explain the use of generated columns.

[MEDIUM] | *Category: Tables | Type: query*

Answer: Generated: auto-derived: CREATE TABLE logs (id INT, full_path VARCHAR(200) GENERATED ALWAYS AS (CONCAT(dir, '/', file)) STORED). VIRTUAL recomputes; STORED persists/indexable. For computed keys (e.g., MD5 hash), JSON paths. Pros: integrity, no triggers; cons: space/update cost. Postgres 12+; use for denormalized views.

Q60. How do you implement full-text search?

[HARD] | *Category: Search | Type: query*

Answer: Full-text indexes text: Postgres CREATE INDEX ON docs USING GIN(to_tsvector('english', content)); SELECT * WHERE to_tsvector(content) @@ to_tsquery('sql & interview'). Scoring with ts_rank. MySQL FULLTEXT MATCH AGAINST. Pros: stemming, relevance; cons: lang-specific, rebuilds. For search apps; integrate Lucene if complex.

Q61. What are table partitions and subpartitions?

[HARD] | *Category: Performance* | *Type: query*

Answer: Partitions split: `CREATE TABLE sales PARTITION BY RANGE (date) (PARTITION p2024 VALUES LESS THAN ('2025-01-01'), ...); SUBPARTITION BY HASH(id) SUBPARTITIONS 4`. Pruning: queries skip. Manage: `EXCHANGE/REBUILD`. Pros: parallel delete/query; cons: cross-part joins slow. For time-series; auto in BigQuery.

Q62. Explain encryption at rest vs in transit.

[HARD] | *Category: Security* | *Type: query*

Answer: At rest: TDE encrypts files/DB: `ALTER DATABASE SET ENCRYPTION ON (cert)`. Protects theft. In transit: TLS/SSL for connections: `Encrypt=True` in conn string. Prevents eavesdropping. At rest static; transit dynamic. Keys: EKM for rotation. Overhead low; PCI/HIPAA req.

Q63. How do you use recursive CTEs for graph traversal?

[HARD] | *Category: CTE* | *Type: query*

Answer: Recursive CTE for graphs: `WITH RECURSIVE path (node, path, depth) AS (SELECT node, ARRAY[node], 1 FROM graph WHERE id=START UNION ALL SELECT g.node, p.path ||| g.node, p.depth+1 FROM graph g JOIN path p ON g.from=p.node WHERE g.node != ALL(p.path) AND depth<MAX) SELECT \* FROM path`. Detects cycles via array. For social networks; index edges.

Q64. What is the NTILE window function?

[MEDIUM] | *Category: Window Functions* | *Type: query*

Answer: `NTILE(n)` divides partition into n groups: `NTILE(5) OVER(ORDER BY revenue)` assigns 1-5 quintiles. Even distribution; last bucket may have more. For segmentation (e.g., RFM analysis). With `PARTITION BY customer`. Ties sequential.

Q65. How do you unpivot data dynamically?

[HARD] | *Category: Data Transformation* | *Type: query*

Answer: `UNPIVOT` cols to rows: `SELECT product, metric, val FROM (SELECT \* FROM pivoted) UNPIVOT (val FOR metric IN (Jan\_sales, Feb\_sales)) u`. Dynamic: `UNION ALL` per col in loop: `SELECT 'Jan' metric, Jan\_sales val FROM t UNION SELECT 'Feb', Feb\_sales FROM t`. For ETL normalization.

Q66. Explain the use of FIRST_VALUE and LAST_VALUE.

[MEDIUM] | *Category: Window Functions* | *Type: query*

Answer: FIRST_VALUE(col) OVER(ORDER BY date ROWS UNBOUNDED PRECEDING) first in partition. LAST_VALUE requires frame: OVER(ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) for last. IGNORE NULLS option. For session starts/ends. Combine with WHERE for filters.

Q67. What is a filtered index?

[MEDIUM] | *Category: Performance* | *Type: query*

Answer: Filtered: CREATE INDEX ON table (col) WHERE status='active'. Indexes subset; smaller/faster for WHERE matches. SQL Server; like partial in Postgres. Use for sparse (e.g., soft deletes). No updates to filtered rows.

Q68. How do you audit database changes?

[MEDIUM] | *Category: Security* | *Type: query*

Answer: Auditing: Triggers INSERT audit VALUES (OLD.id, USER(), NOW(), 'update'). Or SQL Server CDC: sp_cdc_enable_db; query cdc tables. Server audit for logins. Pros: granular; cons: perf (async if possible). Rotate; query for compliance.

Q69. What are common table expressions with multiple CTEs?

[MEDIUM] | *Category: CTE* | *Type: query*

Answer: Multiple: WITH cte1 AS (SELECT dept, AVG(sal) avg FROM emp GROUP BY dept), cte2 AS (SELECT e.*, c.avg FROM emp e JOIN cte1 c ON e.dept=c.dept) SELECT * FROM cte2 WHERE sal > avg. Chained; improves modularity. No inter-deps except sequential. For complex pipelines.

Q70. Explain window frame specifications.

[HARD] | *Category: Window Functions* | *Type: query*

Answer: Frames: ROWS/RANGE/GROUPS BETWEEN bounds. ROWS 2 PRECEDING TO CURRENT: 3 physical rows. RANGE: peers (e.g., same value). Example: AVG(sal) OVER(ORDER BY date ROWS 7 PRECEDING) 7-day moving. Default RANGE to CURRENT. For precise calcs like EMA.

Q71. How do you handle JSON data querying?

[MEDIUM] | *Category: JSON* | *Type: query*

Answer: JSON: Postgres data->'key' extracts; \#> '{path}' nested. Index GIN (jsonb). SELECT * WHERE data @> '{"status":"active"}'. MySQL JSON_SEARCH, ->>. Aggregate: json_agg. For APIs; extract to columns for perf.

Q72. What is connection pooling and its benefits?

[MEDIUM] | *Category: Performance* | *Type: query*

Answer: Pooling reuses connections: HikariCP min 10, max 100; reduces open/close overhead (100ms+). Benefits: throughput up 10x, less resource exhaustion. Monitor idle timeout; leaks via JMX. In cloud: RDS Proxy. Tune for app load.

Q73. Explain role-based access control (RBAC) in SQL.

[MEDIUM] | *Category: Security* | *Type: query*

Answer: RBAC: CREATE ROLE viewer; GRANT SELECT ON SCHEMA public TO viewer; GRANT viewer TO user1. Hierarchical: GRANT role2 TO role1. Vs DAC: scalable for teams. Audit grants; revoke cascades. For orgs, least privilege.

Q74. How do you use CTEs for running totals?

[MEDIUM] | *Category: CTE* | *Type: query*

Answer: WITH ordered AS (SELECT id, val, ROW_NUMBER() OVER(ORDER BY date) rn FROM t) SELECT rn, SUM(val) OVER(ORDER BY rn ROWS UNBOUNDED PRECEDING) running FROM ordered. Or direct window. For inventory; index order col. Vs self-join: O(n).

Q75. What is the PERCENT_RANK window function?

[MEDIUM] | *Category: Window Functions* | *Type: query*

Answer: PERCENT_RANK() OVER(ORDER BY score DESC) = (rank-1)/(total-1), 0-1 relative rank. For % better than: 0.8 top 20%. PARTITION by group. Ties averaged in RANGE. Use in scoring, percentiles.

Q76. How do you pivot with conditional aggregation?

[MEDIUM] | *Category: Data Transformation* | *Type: query*

Answer: Pivot manual: SELECT product, SUM(CASE WHEN region='North' THEN sales ELSE 0 END) North_sales FROM data GROUP BY product. For static cols; dynamic build string. Cross-DB; no operator needed. For Excel-like reports.

Q77. Explain the use of HASH vs MERGE joins.

[HARD] | *Category: Performance* | *Type: query*

Answer: HASH: builds in-memory hash on build side (smaller table), probes; equi-joins, no sort. MERGE: sorted inputs, zipper; preserves order, range scans. Optimizer picks; bad stats → hint. HASH spills to tempdb if large. For big data.

Q78. What are database sequences and their advantages?

[EASY] | *Category: Sequences* | *Type: query*

Answer: Sequences: `CREATE SEQUENCE order\_seq;` `NEXTVAL(order\_seq)` unique IDs. Advantages: concurrent (no locks like auto-inc), shareable, `CACHE 20` for perf. Cycle for wrap. Postgres/Oracle; vs identity: gap-tolerant. For sharding.

Q79. How do you secure stored procedures?

[MEDIUM] | *Category: Security | Type: query*

Answer: Secure: WITH EXECUTE AS OWNER for context; validate params (IF @id<=0 RETURN); parameterized dynamic SQL; schema-bind if UDF. GRANT EXECUTE TO role only. No xp_cmdshell. Audit executions. Prevent elevation.

Q80. Explain CTE materialization.

[HARD] | *Category: CTE | Type: query*

Answer: CTEs inline or temp table per optimizer: large/reused → materialize. Postgres WITH ... AS MATERIALIZED forces. Vs temp: no DDL. Recursive always temp. Check plan: Table Scan on CTE. For perf, use temp if scanned multiple.

Q81. What is CUME_DIST window function?

[MEDIUM] | *Category: Window Functions | Type: query*

Answer: CUME_DIST() OVER(ORDER BY val) = rows <= current / total, cumulative % up to row. For CDF: 0.6 means 60% at/below. RANGE includes ties. Use in distributions, thresholds.

Q82. How do you handle data skew in joins?

[HARD] | *Category: Performance | Type: query*

Answer: Skew: few keys dominate; detect via histograms (ANALYZE). Mitigate: broadcast small side, salt keys (add rand%100), separate skew tables, dynamic partitioning. In SQL: UNION skewed/normal. For hash joins, avoid OOM.

Q83. What is transparent data encryption (TDE)?

[HARD] | *Category: Security | Type: query*

Answer: TDE: whole-DB encryption: CREATE DATABASE ENCRYPTION KEY WITH ALGORITHM AES_256 ENCRYPTED BY cert; ALTER DB SET ENCRYPTION ON. Transparent; protects mdf/ldf. Backups encrypted. Keys exportable; EKM for cloud. ~3-5% overhead. For rest compliance.

Q84. How do you use CTEs for data validation?

[MEDIUM] | *Category: CTE | Type: query*

Answer: WITH anomalies AS (SELECT * FROM orders EXCEPT SELECT * FROM expected_orders)
SELECT COUNT(*) FROM anomalies. Or recursive for referential chains. If >0, raise error. For ETL:
assert integrity. Vs EXCEPT direct: modular.

Q85. Explain the ROWS vs RANGE in window frames.

[HARD] | *Category: Window Functions* | *Type: query*

Answer: ROWS: count physical rows (e.g., ROWS 1 PRECEDING: prior row). RANGE: value-based (RANGE 1 DAY PRECEDING: same/prior day). RANGE peers ties; for time windows. Example: SUM OVER(RANGE INTERVAL 1 DAY). Default RANGE.

Q86. What are partial indexes?

[MEDIUM] | *Category: Performance* | *Type: query*

Answer: Partial: CREATE INDEX ON logs (user_id) WHERE deleted=false. Indexes active subset; tiny for flags. Postgres; filtered equiv SQL Server. Use for 90% filtered queries. No index on false rows.

Q87. How do you prevent SQL injection in dynamic queries?

[MEDIUM] | *Category: Security* | *Type: query*

Answer: Parameterized: EXEC sp_executesql N'SELECT * FROM users WHERE id=@id', @id=5. Or PREPARE/EXECUTE in MySQL. No concat strings. Validate/escape if needed. ORM auto-params. Audit dynamic use.

Q88. What is a hierarchical query without recursion?

[HARD] | *Category: Hierarchical Queries* | *Type: query*

Answer: Oracle CONNECT BY: SELECT LEVEL, name FROM emp START WITH manager IS NULL CONNECT BY PRIOR id=manager_id. Pseudos: LEVEL, SYS_CONNECT_BY_PATH. For trees; limited depth. Vs CTE: vendor-specific. Modern migrate to recursive.

Q89. Explain the use of LEAD with offset.

[MEDIUM] | *Category: Window Functions* | *Type: query*

Answer: LEAD(col, 3, 'N/A') OVER(PARTITION BY dept ORDER BY date) third future value. For forecasts: sales - LEAD(sales,1). Offset dynamic via var. Default end. For gaps/trends.

Q90. How do you optimize for write-heavy workloads?

[HARD] | *Category: Performance* | *Type: query*

Answer: Writes: bulk INSERT (BULK/VALUES), minimal indexes (disable/re-enable), staging tables, batched UPDATE, async logging. Tune checkpoints/WAL. Partition append-only. Vs reads: less denorm. Monitor I/O waits.

Q91. What are database roles and schemas?

[MEDIUM] | *Category: Security* | *Type: query*

Answer: Roles group perms: CREATE ROLE auditor; GRANT SELECT ON audit.* TO auditor. Schemas namespace: CREATE SCHEMA prod; objects in prod.table. GRANT USAGE/ALL ON SCHEMA. Vs users: inheritable. For deploys.

Q92. How do CTEs interact with temp tables?

[MEDIUM] | *Category: CTE* | *Type: query*

Answer: CTEs reference temps: WITH cte AS (SELECT * FROM \#temp) SELECT * FROM cte. Temps persist; CTEs ephemeral. For hybrid: load temp, CTE transform. Optimizer: CTE may spill to temp. Choose: temp for indexes/multi-session.

Q93. What is the LAG function for gap detection?

[MEDIUM] | *Category: Window Functions* | *Type: query*

Answer: LAG(date,1) OVER(ORDER BY id); WHERE date > LAG(date,1) OVER(ORDER BY id) +1 detects missing. COUNT gaps: SUM(CASE WHEN gap THEN 1 ELSE 0 END) OVER(). For sequences, logs.

Q94. Explain index fragmentation and defragmentation.

[HARD] | *Category: Performance* | *Type: query*

Answer: Fragmentation: splits >30% (avg_fragmentation_in_percent). Detect: sys.dm_db_index_physical_stats. Defrag: ALTER INDEX REORGANIZE (low frag) or REBUILD (high, online=ON). Schedule weekly; fillfactor 70 OLTP. Vs drop/create.

Q95. How do you implement data masking?

[MEDIUM] | *Category: Security* | *Type: query*

Answer: Masking: Dynamic Data Masking (SQL Server): ALTER COLUMN email ADD MASKED WITH (FUNCTION='email()'). Views: CASE WHEN @role='admin' THEN email ELSE LEFT(email,3)+'***' END. Static for non-prod: UPDATE SET masked=SHA2(email). For PII in dev.

Q96. What are recursive queries for bill of materials?

[HARD] | *Category: CTE* | *Type: query*

Answer: BOM explosion: WITH RECURSIVE bom(part, component, qty) AS (SELECT part, sub, 1 FROM assembly WHERE part=TOP UNION ALL SELECT a.part, a.sub, bom.qty*a.qty FROM assembly a JOIN bom ON a.part=bom.component) SELECT SUM(qty) FROM bom GROUP BY part. Accumulates; cycle detect. For supply chain.

Q97. How does the RANK function handle ties?

[EASY] | *Category: Window Functions* | *Type: query*

Answer: RANK() ties share (1,1), skips next (3). OVER(ORDER BY sal DESC). For positions with gaps. Vs DENSE no skip. NULLs low/high? DESC high. Use in leaderboards.

Q98. What is adaptive query processing?

[HARD] | *Category: Performance* | *Type: query*

Answer: Adaptive: runtime plan adjust, e.g., SQL Server batch serialization, memory grant feedback (resizes on second exec). For variable cardinality. Reduces spills; in Azure. Vs static: handles drift. Monitor via Query Store.

Q99. Explain certificate-based authentication.

[HARD] | *Category: Security* | *Type: query*

Answer: Cert auth: CREATE CERTIFICATE client_cert; CREATE LOGIN FROM CERTIFICATE client_cert; map to users. Client presents cert on connect. Stronger than passwords; for services. Revoke by dropping cert. Azure AD hybrid.

Q100. How do you use CTEs for pagination?

[MEDIUM] | *Category: CTE* | *Type: query*

Answer: WITH ranked AS (SELECT *, ROW_NUMBER() OVER(ORDER BY id) rn FROM large_table WHERE filter) SELECT * FROM ranked WHERE rn BETWEEN @offset+1 AND @offset+@limit. Efficient post-filter rank. Index order; vs OFFSET direct (slower deep).

Q101. What is the DENSE_RANK function for leaderboards?

[MEDIUM] | *Category: Window Functions* | *Type: query*

Answer: DENSE_RANK() ties share (1,1), next consecutive (2). No gaps; for dense packing. PARTITION by league. For tournaments without skips.

Q102. How do you tune for concurrent users?

[HARD] | *Category: Performance* | *Type: query*

Answer: Concurrency: SNAPSHOT isolation, rowversion optimistic, covering indexes, read scaled replicas. Tune maxdop=1, cost threshold. Monitor SOS\SCHEDULER\YIELD waits. App: pooling, async IO. Shard if extreme.

Q103. What are audit specifications in SQL Server?

[MEDIUM] | *Category: Security* | *Type: query*

Answer: Audit specs: CREATE SERVER AUDIT SPECIFICATION s FOR SERVER AUDIT a ADD (DATABASE_OPERATION_GROUP); Logs to file/event. Filter events/users. For SOX: SELECT/INSERT on sensitive. Vs trace: lighter. Query fn_dblog.

Q104. Explain non-recursive CTEs for modularity.

[EASY] | *Category: CTE* | *Type: query*

Answer: Non-rec: WITH filters AS (SELECT * FROM raw WHERE date>=@start), aggs AS (SELECT dept, SUM(sal) FROM filters GROUP BY dept) SELECT * FROM aggs ORDER BY sum DESC. Stepwise; reference prior CTEs. For refactoring monsters.

Q105. How do you use window functions for cohort analysis?

[HARD] | *Category: Window Functions* | *Type: query*

Answer: Cohort retention: FIRST_VALUE(signup_month) OVER(PARTITION BY user ORDER BY activity_date) cohort, COUNT(*) OVER(PARTITION BY user, cohort_month_diff) / COUNT(*) OVER(PARTITION BY cohort, cohort_month_diff) rate. For churn.

Q106. What is statistics maintenance?

[MEDIUM] | *Category: Performance* | *Type: query*

Answer: Stats: UPDATE STATISTICS table (SAMPLE 10 PERCENT); histograms for optimizer. Auto SQL Server (trace 2028); manual after bulk. Fullscan accurate but slow. For skewed: auto-update on 500+ mods.

Q107. How do you secure data in transit?

[MEDIUM] | *Category: Security* | *Type: query*

Answer: Transit: Enforce TLS: ALTER SERVER CONFIGURATION SET ENCRYPTION = ON; client VerifyCertificate. TDS EncryptedConnection. Vs VPN: app-level. Cert pinning prevents MITM. Audit unencrypted connects.

Q108. What are the limitations of CTEs?

[MEDIUM] | *Category: CTE* | *Type: query*

Answer: Limits: single-statement scope, no indexes, potential non-materialization (re-execute), update restrictions (no TOP/OLAP). Depth 100 recursive. Vs views: no persist. For ultra-complex: procs/temps.

Q109. Explain the use of ROW_NUMBER for deduplication.

[MEDIUM] | *Category: Window Functions* | *Type: query*

Answer: Deduplicate: WITH dups AS (SELECT *, ROW_NUMBER() OVER(PARTITION BY email ORDER BY created DESC) rn FROM users) DELETE FROM dups WHERE rn>1. Keeps newest. Safe in trans; index partition. Vs MIN(id) subquery.

Q110. What is conditional aggregation in SQL?

[MEDIUM] | *Category: Aggregations* | *Type: query*

Answer: Conditional aggregation applies CASE inside aggregates for conditional sums/counts, e.g., SELECT dept, SUM(CASE WHEN gender='M' THEN salary ELSE 0 END) AS male_salary FROM employees GROUP BY dept. It pivots data without PIVOT, efficient for reports. Handles NULLs via COALESCE; outperforms self-joins for large datasets. Use for gender-based analytics or status filtering. In 2025, key for BI tools like Tableau integration. Alternatives: filtered aggregates in Postgres 14+.

Q111. Write a SQL query to select all employees from the employees table where the salary is greater than 50000.

[EASY] | *Category: SELECT* | *Type: query*

Answer: SELECT * FROM employees WHERE salary > 50000;

Q112. How do you find the second highest salary from the employees table?

[MEDIUM] | *Category: AGGREGATE* | *Type: query*

Answer: SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);

Q113. Write a query to join two tables, employees and departments, on department_id.

[EASY] | *Category: JOIN* | *Type: query*

Answer: SELECT e.name, d.department_name FROM employees e INNER JOIN departments d ON e.department_id = d.id;

Q114. Delete all duplicate records from a table keeping only one instance of each duplicate.

[HARD] | *Category: DELETE | Type: query*

Answer: DELETE FROM table_name WHERE id NOT IN (SELECT MIN(id) FROM table_name GROUP BY column_name HAVING COUNT(*) > 1);

Q115. Write a SQL query to find the number of employees in each department.

[MEDIUM] | Category: GROUP BY | Type: query

Answer: SELECT department_id, COUNT(*) as employee_count FROM employees GROUP BY department_id;

Q116. How to update the salary of all employees in a specific department by 10%?

[EASY] | Category: UPDATE | Type: query

Answer: UPDATE employees SET salary = salary * 1.10 WHERE department_id = 5;

Q117. Write a query to select the top 5 highest paid employees.

[EASY] | Category: ORDER BY | Type: query

Answer: SELECT * FROM employees ORDER BY salary DESC LIMIT 5;

Q118. Find employees who earn more than their department average salary.

[HARD] | Category: SUBQUERY | Type: query

Answer: SELECT e1.* FROM employees e1 WHERE e1.salary > (SELECT AVG(e2.salary) FROM employees e2 WHERE e2.department_id = e1.department_id);

Q119. Write a query to insert a new employee into the employees table.

[EASY] | Category: INSERT | Type: query

Answer: INSERT INTO employees (name, salary, department_id) VALUES ('John Doe', 60000, 1);

Q120. How to create a view that shows employee names and their manager names?

[MEDIUM] | Category: VIEW | Type: query

Answer: CREATE VIEW employee_managers AS SELECT e.name as employee, m.name as manager FROM employees e LEFT JOIN employees m ON e.manager_id = m.id;

Q121. Write a query to find the employee with the highest salary in each department.

[MEDIUM] | Category: WINDOW FUNCTION | Type: query

Answer: SELECT department_id, MAX(salary) as max_salary FROM employees GROUP BY department_id;

Q122. How do you handle NULL values in a SELECT statement?

[EASY] | Category: NULL HANDLING | Type: query

Answer: SELECT COALESCE(column_name, 'Default Value') FROM table_name;

Q123. Write a query to find duplicates in the email column of users table.

[MEDIUM] | Category: DUPLICATES | Type: query

Answer: SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

Q124. Create a recursive query to fetch all subordinates of a manager.

[HARD] | Category: RECURSIVE CTE | Type: query

Answer: WITH RECURSIVE subordinates AS (SELECT id, name, manager_id FROM employees WHERE manager_id = 1 UNION ALL SELECT e.id, e.name, e.manager_id FROM employees e INNER JOIN subordinates s ON e.manager_id = s.id) SELECT * FROM subordinates;

Q125. How to pivot data in SQL to transform rows into columns?

[HARD] | Category: PIVOT | Type: query

Answer: SELECT department_id, SUM(CASE WHEN salary < 50000 THEN 1 ELSE 0 END) as low_salary, SUM(CASE WHEN salary >= 50000 THEN 1 ELSE 0 END) as high_salary FROM employees GROUP BY department_id;

Q126. Find the Nth highest salary using window functions.

[MEDIUM] | Category: WINDOW FUNCTION | Type: query

Answer: SELECT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) as rnk FROM employees) WHERE rnk = N;

Q127. Write a query to find the difference between consecutive rows in a table.

[MEDIUM] | *Category: WINDOW FUNCTION | Type: query*

Answer: SELECT time, value, value - LAG(value) OVER (ORDER BY time) as diff FROM sales;

Q128. How to find the running total for sales by date?

[MEDIUM] | *Category: WINDOW FUNCTION | Type: query*

Answer: SELECT date, sales, SUM(sales) OVER (ORDER BY date) as running_total FROM sales_table;

Q129. Query to find customers who placed more than 3 orders.

[EASY] | *Category: GROUP BY | Type: query*

Answer: SELECT customer_id FROM orders GROUP BY customer_id HAVING COUNT(*) > 3;

Q130. Write a LEFT JOIN query to find employees without departments.

[MEDIUM] | *Category: JOIN | Type: query*

Answer: SELECT e.* FROM employees e LEFT JOIN departments d ON e.dept_id = d.id WHERE d.id IS NULL;

Q131. How to use UNION to combine results from two SELECT statements?

[EASY] | *Category: SET OPERATIONS | Type: query*

Answer: SELECT name FROM table1 UNION SELECT name FROM table2;

Q132. Find the average salary per department using GROUP BY.

[MEDIUM] | *Category: AGGREGATE | Type: query*

Answer: SELECT department_id, AVG(salary) FROM employees GROUP BY department_id;

Q133. Write a query to replace employee ID with unique identifier from another table.

[EASY] | Category: JOIN | Type: query

Answer: SELECT u.unique_id, e.name FROM Employees e LEFT JOIN EmployeeUNI u ON e.id = u.id;

Q134. How to select recyclable and low-fat products?

[EASY] | Category: SELECT | Type: query

Answer: SELECT product_id FROM Products WHERE low_fats = 'Y' AND recyclable = 'Y';

Q135. Find customers who are not referred by customer with id 2.

[EASY] | Category: WHERE | Type: query

Answer: SELECT name FROM Customer WHERE referee_id != 2 OR referee_id IS NULL;

Q136. Select big countries with area > 3000000 or population > 25000000.

[EASY] | Category: WHERE | Type: query

Answer: SELECT name, population, area FROM World WHERE area > 3000000 OR population > 25000000;

Q137. Find articles viewed by author id 1.

[EASY] | Category: DISTINCT | Type: query

Answer: SELECT DISTINCT author_id AS id FROM Views WHERE author_id = 1;

Q138. Select invalid tweets longer than 15 characters.

[EASY] | Category: STRING FUNCTIONS | Type: query

Answer: SELECT tweet_id FROM Tweets WHERE LENGTH(content) > 15;

Q139. Write a query for product sales analysis.

[EASY] | Category: JOIN | Type: query

Answer: SELECT product_id, year, quantity, price FROM Sales s JOIN Product p ON s.product_id = p.product_id;

Q140. How to use CTE for complex queries?

[MEDIUM] | Category: CTE | Type: query

Answer: WITH cte AS (SELECT * FROM table) SELECT * FROM cte WHERE condition;

Q141. Find the percentage of users who are active.

[MEDIUM] | Category: AGGREGATE | Type: query

Answer: SELECT ROUND((COUNT(CASE WHEN active = 1 THEN 1 END) * 100.0 / COUNT(*)), 2) FROM users;

Q142. Query to find the mode (most frequent value) in a column.

[HARD] | Category: AGGREGATE | Type: query

Answer: SELECT mode_column FROM (SELECT column, COUNT(*) as cnt FROM table GROUP BY column ORDER BY cnt DESC LIMIT 1) t;

Q143. How to handle date ranges in queries?

[EASY] | Category: DATE FUNCTIONS | Type: query

Answer: SELECT * FROM orders WHERE order_date BETWEEN '2025-01-01' AND '2025-12-31';

Q144. Write a query using LIKE for pattern matching.

[EASY] | Category: LIKE | Type: query

Answer: SELECT * FROM customers WHERE name LIKE 'A%';

Q145. Find the first and last login for each user.

[MEDIUM] | Category: GROUP BY | Type: query

Answer: SELECT user_id, MIN(login_time) as first, MAX(login_time) as last FROM logins GROUP BY user_id;

Q146. How to use CASE statement for conditional logic?

[EASY] | Category: CASE | Type: query

Answer: SELECT name, CASE WHEN salary > 50000 THEN 'High' ELSE 'Low' END as category FROM employees;

Q147. Query to swap gender in a table.

[MEDIUM] | Category: UPDATE | Type: query

Answer: UPDATE salary SET sex = CASE WHEN sex = 'm' THEN 'f' ELSE 'm' END;

Q148. Find consecutive numbers in a table.

[HARD] | Category: JOIN | Type: query

Answer: SELECT DISTINCT l1.num AS ConsecutiveNums FROM Logs l1 JOIN Logs l2 ON l1.id = l2.id + 1 AND l1.num = l2.num JOIN Logs l3 ON l2.id = l3.id + 1 AND l2.num = l3.num;

Q149. Write a query for self-join to find employees earning more than managers.

[MEDIUM] | Category: JOIN | Type: query

Answer: SELECT e1.name FROM employee e1 JOIN employee e2 ON e1.managerId = e2.id WHERE e1.salary > e2.salary;

Q150. How to delete duplicate emails?

[MEDIUM] | Category: DELETE | Type: query

Answer: DELETE p1 FROM Person p1 JOIN Person p2 ON p1.email = p2.email AND p1.id > p2.id;

Q151. Find the rank of each employee by salary.

[MEDIUM] | Category: WINDOW FUNCTION | Type: query

Answer: SELECT name, salary, DENSE_RANK() OVER (ORDER BY salary DESC) as rank FROM employees;

Q152. Query to find overlapping intervals.

[HARD] | Category: SUBQUERY | Type: query

Answer: SELECT * FROM meetings m1 WHERE EXISTS (SELECT 1 FROM meetings m2 WHERE m1.start < m2.end AND m1.end > m2.start AND m1.id != m2.id);

Q153. How to use FULL OUTER JOIN?

[MEDIUM] | Category: JOIN | Type: query

Answer: SELECT * FROM table1 FULL OUTER JOIN table2 ON table1.id = table2.id;

Q154. Write a query for cohort analysis.

[HARD] | Category: DATE FUNCTIONS | Type: query

Answer: SELECT cohort_month, COUNT(*) as users FROM (SELECT DATE_TRUNC('month', signup_date) as cohort_month FROM users) GROUP BY cohort_month;

Q155. Find the median salary.

[HARD] | Category: AGGREGATE | Type: query

Answer: SELECT AVG(salary) as median FROM (SELECT salary FROM employees ORDER BY salary LIMIT 2 - (SELECT COUNT(*) FROM employees) % 2 OFFSET (SELECT (COUNT(*) - 1) / 2 FROM employees)) t;

Q156. How to use EXCEPT to find differences between two sets?

[MEDIUM] | Category: SET OPERATIONS | Type: query

Answer: SELECT * FROM table1 EXCEPT SELECT * FROM table2;

Q157. Query to find top 3 salaries per department.

[HARD] | *Category: WINDOW FUNCTION | Type: query*

Answer: `SELECT * FROM (SELECT d.name as dep, e.name, e.salary, DENSE_RANK() OVER (PARTITION BY e.departmentId ORDER BY e.salary DESC) as rnk FROM employee e JOIN department d ON e.departmentId = d.id) WHERE rnk <= 3;`

Q158. How to create an index on a table?

[EASY] | *Category: INDEX | Type: query*

Answer: `CREATE INDEX idx_name ON table(column);`

Q159. Write a query using JSON functions to extract data.

[MEDIUM] | *Category: JSON | Type: query*

Answer: `SELECT json_column->>'key' FROM table;`

Q160. Find users who have never made a purchase.

[MEDIUM] | *Category: JOIN | Type: query*

Answer: `SELECT u.name FROM Users u LEFT JOIN Orders o ON u.id = o.buyer_id WHERE o.buyer_id IS NULL;`